Instytut Telekomunikacji Multimedialnej

Laboratorium 8
Verilog – Liczby zespolone

Data: \2021

4. Czy układ działa poprawnie? Zweryfikować, czy zadanie błędnych wektorów testowych (pliki input.txt i output.txt) wygeneruje stosowny komunikat na konsoli i ustawi flagę "error" na 1. (3p)

Tak, układ działa poprawnie.
Podczas wystąpienia błędu wyświetla flagę error w następnym takcie.

5. Sprawdzenie modułu przez prowadzącego. (7p)

```
module _8_mytestbenchmodule();
reg CLK;
initial CLK <= 0;
always #50 CLK <= ~CLK;
reg RST;
initial
begin
RST <= 0;
RST <= #50 1;
RST <= #500 0;
end
integer fi,fo;
integer count;
reg error;
integer f1, f2, f3, f4;
reg istb;
reg [31:0] vec_Are, vec_Aim, vec_Bre, vec_Bim, vec_Cre, vec_Cim;
wire ostb, i_ack, i_bsy;
wire [31:0] Cre, Cim;
reg o_ack;
initial
begin
fi = $fopen("input.txt","r");
fo = $fopen("output.txt","r");
istb = 0;
error = 0;
o_ack = 0;
end
always @(posedge CLK or posedge RST)
if (!RST & !i_bsy & !ostb) begin
count = $fscanf(fi, "%d %d %d %d", f1,f2,f3,f4); // vec_Are, vec_Aim, vec_Bre, vec_Bim
if (count==4)
begin
istb <= 1;
vec_Are <= f1;
vec_Aim <= f2;
vec_Bre <= f3;
vec_Bim <= f4
#200;
istb <= 0;
end else istb<=0;
end
```

```
always @(posedge ostb) begin
#300;
o_ack = 1;
#100;
o_ack = 0;
end
always @(posedge ostb) begin
count = $fscanf(fo, "%d %d", vec_Cre, vec_Cim);
if (count==2) begin
error = 0;
if ((vec_Cre!=Cre) || (vec_Cim!=Cim)) begin
$display("error %d!=%d %d!=%d", vec_Cre,Cre,vec_Cim,Cim);
error = 1;
end
end
end
```

```
complex_mul cmul
(
.CLK(CLK),
.RST(RST),
.I_STB(istb),
.I_ARE(vec_Are),
.I_AIM(vec_Aim),
.I_BRE(vec_Bre),
.I_BIM(vec_Bim),
.O_STB(ostb),
.O_CRE(Cre),
.O_CIM(Cim),
.I_ACK(i_ack),
.O_ACK(o_ack),
.I_BSY(i_bsy)
);
Endmodule
module adder
#( parameter A_WIDTH = 32,
parameter B_WIDTH = 32)
( input wire RST,
input wire CLK,
input wire I_STB,
output wire I_ACK,
input wire [A_WIDTH-1:0] I_DAT_A,
input wire [B_WIDTH-1:0] I_DAT_B,
output reg O_STB,
output reg [(A_WIDTH>B_WIDTH?A_WIDTH:B_WIDTH) : 0] O_DAT,
input wire O_ACK
);
assign I_ACK = I_STB & ~O_STB;
always @(posedge CLK or posedge RST)
if (RST) O_DAT <= 0;
else if (I_ACK) O_DAT <= I_DAT_A+I_DAT_B;
always @(posedge CLK or posedge RST)
if (RST) O_STB <= 0;
else if (O_ACK) O_STB <= 0;
else if (I_ACK) O_STB <= 1;
endmodule
module complex_mul
(
```

```
42
43  input wire CLK,
44  input wire RST,
45  input wire I_STB,
46  input wire [31:0] I_ARE,
47  input wire [31:0] I_AIM,
48  input wire [31:0] I_BRE,
49  input wire [31:0] I_BIM,
50  output wire O_STB,
51  output wire [31:0] O_CRE,
52  output wire [31:0] O_CIM,
53  output wire I_ACK,
54  input wire O_ACK,
55  output wire I_BSY,
56  );
57
58  wire [63:0] AreBre;
59  wire [63:0] AimBim;
60  wire [63:0] AimBre;
61  wire [63:0] AreBim;
62  wire [32:0] AimBimNegative;
63  wire [32:0] o_cre;
64  wire [32:0] o_cim;
65  wire stb1;
66  wire stb2;
67  wire ack1;
68  wire ack2;
69  wire ack3;
70  wire ack4;
71  reg i_bsy;
72  always @(posedge RST) i_bsy = 0;
73  always @(posedge I_ACK) i_bsy <= 1;
74  always @(posedge O_ACK) if (O_STB) i_bsy <= 0;
75  assign O_CRE = o_cre[31:0];
76  assign O_CIM = o_cim[31:0];
77  assign I_ACK = I_STB & !ack4;
78  assign I_BSY = i_bsy;
79  multiplier _AreBre
80  (
81  .CLK(CLK),
82  .RST(RST),
83  .I_DAT_A(I_ARE),
```

```
 84    .I_DAT_B(I_BRE),
 85    .I_STB(I_STB),
 86    .I_ACK(ack4),
 87    .O_DAT(AreBre),
 88    .O_STB(stb1),
 89    .O_ACK(ack3)
 90    );
 91    multiplier _AimBim
 92    (
 93    .CLK(CLK),
 94    .RST(RST),
 95    .I_DAT_A(I_AIM),
 96    .I_DAT_B(I_BIM),
 97    .I_STB(I_STB),
 98
 99    .I_ACK(),
100    .O_DAT(AimBim),
101    .O_STB(),
102    .O_ACK(ack3)
103    );
104    multiplier _AimBre
105    (
106    .CLK(CLK),
107    .RST(RST),
108    .I_DAT_A(I_AIM),
109    .I_DAT_B(I_BRE),
110    .I_STB(I_STB),
111    .I_ACK(),
112    .O_DAT(AimBre),
113    .O_STB(),
114    .O_ACK(ack1)
115    );
116    multiplier _AreBim
117    (
118    .CLK(CLK),
119    .RST(RST),
120    .I_DAT_A(I_ARE),
121    .I_DAT_B(I_BIM),
122    .I_STB(I_STB),
123    .I_ACK(),
124    .O_DAT(AreBim),
125    .O_STB(),
```

```
126     .O_ACK(ack1)
127     );
128     adder makeAimBimNegative
129     (
130     .CLK(CLK),
131     .RST(RST),
132     .I_DAT_A(~AimBim[31:0]),
133     .I_DAT_B(32'b1),
134     .I_STB(stb1),
135     .I_ACK(ack3),
136     .O_DAT(AimBimNegative),
137     .O_STB(stb2),
138     .O_ACK(ack2)
139     );
140     adder AreBreMinusAimBim
141     (
142     .CLK(CLK),
143     .RST(RST),
144     .I_DAT_A(AreBre[31:0]),
145     .I_DAT_B(AimBimNegative[31:0]),
146     .I_STB(stb2),
147     .I_ACK(ack2),
148     .O_DAT(o_cre),
149     .O_STB(),
150     .O_ACK(O_ACK)
151     );
152     adder AimBrePlusAreBim
153     (
154     .CLK(CLK),
155     .RST(RST),
156     .I_DAT_A(AimBre[31:0]),
157     .I_DAT_B(AreBim[31:0]),
158     .I_STB(stb2),
159     .I_ACK(ack1),
160
161     O_DAT(o_cim),
162     .O_STB(O_STB),
163     .O_ACK(O_ACK)
164     );
165     endmodule
166     module multiplier
167     #(
```

```
168  parameter A_WIDTH = 32,
169  parameter B_WIDTH = 32
170  )
171  (
172  input wire RST,
173  input wire CLK,
174  input wire I_STB,
175  output wire I_ACK,
176  input wire [A_WIDTH-1:0] I_DAT_A,
177  input wire [B_WIDTH-1:0] I_DAT_B,
178  output reg O_STB,
179  output reg [(A_WIDTH + B_WIDTH)-1 : 0] O_DAT,
180  input wire O_ACK
181  );
182  assign I_ACK = I_STB & ~O_STB;
183  always @(posedge CLK or posedge RST)
184  if (RST) O_DAT <= 0;
185  else if (I_ACK) O_DAT <= I_DAT_A*I_DAT_B;
186  always @(posedge CLK or posedge RST)
187  if (RST) O_STB <= 0;
188  else if (O_ACK) O_STB <= 0;
189  else if (I_ACK) O_STB <= 1;
190  endmodule
```